

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FLIGHT TICKET BOOKING

Network Programming: Application Protocol Design

Presented by:

Flight Ticket Project

Design review 6 November 2026

ONE LOVE. ONE FUTURE.

Application problem and scope

Customer goal

- Search several airlines and compare price and flight duration.
- Reserve seats, pay, receive ticket codes and manage bookings.
- Receive flight updates even after reconnecting.

Project scope

- Mock domestic flights and account-balance payment.
- Customer and admin clients use direct sockets.
- Real e-ticket email only. SMS is excluded.

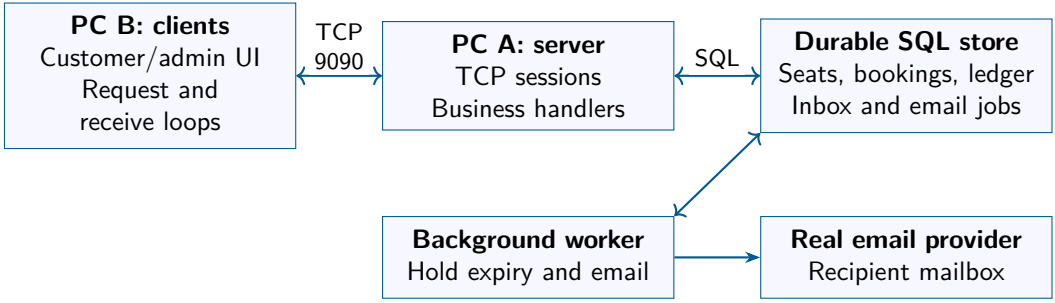
Design objective: keep seats, booking status, balance and ticket records consistent when requests fail or connections drop.

Required functionality and grading coverage

Requirement	Observable application behavior	Points
R1 Sockets	Client and server communicate over a custom socket protocol	2
R2 Logging	Server records received and sent messages	1
R3 Accounts	Register, authenticate and log out	2
R4 Search	Route, departure/return date, passengers and seat class	2
R5 Compare	Airline prices and flight duration	2
R6 Purchase	Hold seats, check balance, pay and issue tickets	3
R7 E-ticket	Send ticket codes through real email	2
R8 Management	View, cancel, change and print tickets	3
R9 Notifications	Admin triggers schedule, cancellation and delay updates	3

Total: 20 points. Protocol design supports every function.

Client–server architecture



Server: C, C++ or Java with direct socket handling. Multiple clients connect concurrently. Server logs track receipt, rejection and send outcomes.

Customer and admin procedures

Actor	Procedure	Client/server responsibility
Guest	Register, then log in	Server creates USER and binds identity to the connection
Customer	Search, compare, book, pay	Client selects flights. Server prices, holds and debits
Customer	Receive email and manage tickets	Server returns current booking/ticket data. Client prints locally
Customer	Review flight notifications	Client handles pushes and retrieves unread inbox entries
Admin	List flights, delay, reschedule, cancel	Server checks ADMIN, updates affected bookings and saves notifications

Access rule: customers see only their own bookings and notifications. Registration can never create an admin.

Search: HAN to SGN, 20 December 2026, one adult, ECONOMY.

Flight	Airline	Fare (VND)	Duration
F1002 / VJ131	VietJet Air	1,290,000	130 min
F1003 / QH201	Bamboo Airways	1,590,000	125 min
F1001 / VN213	Vietnam Airlines	1,850,000	130 min

- Search also accepts return date, passenger count, class and sorting.
- Compare 2–5 flights on the same route/date/class. Mark cheapest and fastest.
- Exclude unavailable flights or insufficient seats. Search and comparison hold no seats.

TCP framing and transport rules

One message: one UTF-8 JSON object followed by LF.

```
{"v":1,"type":"PING","id":"p1","data":{}}\n
```

Condition	Required receiver/sender behavior
Partial read	Keep bytes until LF completes the frame
Several frames in one read	Process complete lines in order, retain the unfinished suffix
Partial write	Continue until the entire frame and LF have been sent
Frame limit	At most 65,536 payload bytes, excluding LF and optional CR
Timing	Incomplete frame: 10 s. Idle receive: 300 s. Client request: 10 s

One client request may wait at a time. Serialize complete outgoing frames.

Request and matching response

```
{"v":1,"type":"PING","id":"p1","data":{}}
{"v":1,"type":"PING_RESP","id":"p1","ok":true,
 "code":"OK","data":{"serverTime":"2026-11-04T10:00:00+07:00"}}
```

Failure reply

```
{"v":1,"type":"LOGIN_RESP","id":"l1","ok":false,
 "code":"E_BAD_CREDENTIALS","message":"Invalid credentials","data":{}}
```

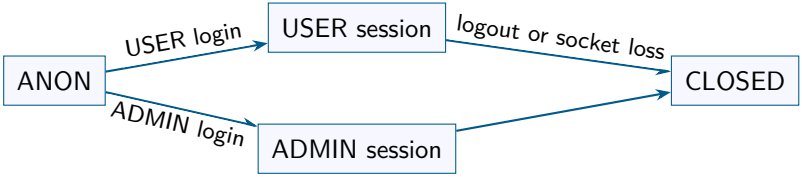
- `v`: protocol version. `type`: operation. `id`: request correlation.
- `NOTIFY` uses `id:null` and has no `ok` field.
- Clients dispatch by type/ID. A push does not complete a pending request.

Protocol operation inventory

Group	Request types
Session and account	`PING`, `REGISTER`, `LOGIN`, `LOGOUT`, `GET_BALANCE`
Search and comparison	`SEARCH_FLIGHTS`, `COMPARE_FLIGHTS`
Booking and payment	`BOOK_FLIGHT`, `PAY_BOOKING`, `RESEND_TICKET`
Ticket management	`LIST_BOOKINGS`, `GET_BOOKING`, `CANCEL_BOOKING`, `CHANGE_BOOKING`, `PRINT_TICKET`
Notifications	`GET_NOTIFICATIONS`, `ACK_NOTIFICATION`
Admin	`ADMIN_LIST_FLIGHTS`, `ADMIN_UPDATE_FLIGHT`

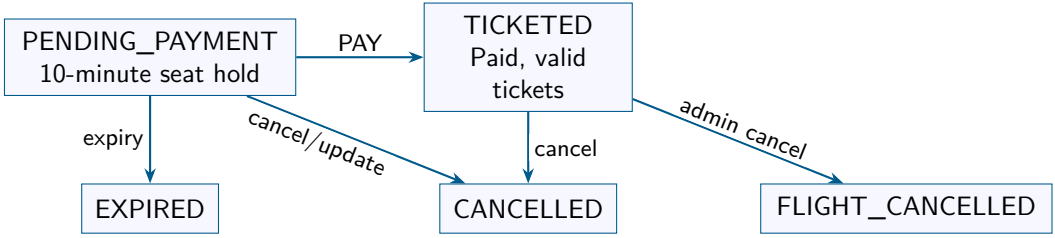
19 request types, typed responses and one unsolicited server message: `NOTIFY`.

Session and client request states



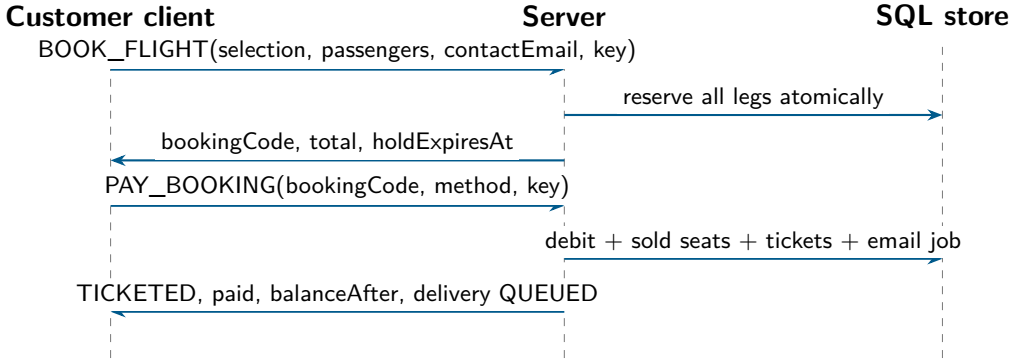
- REGISTER and failed LOGIN leave the connection ANON.
- Role and identity come from the server's stored account, never request fields.
- Client request states: READY, WAITING and UNKNOWN OUTCOME.
- During WAITING, handle `NOTIFY` and continue waiting for the matching response.
- Reconnect starts ANON. Authenticate again before recovering an uncertain operation.

Booking state machine



CHANGE_BOOKING keeps TICKETED and increments revision. Terminal bookings remain viewable, but their tickets are invalid for print/resend.

Booking and payment message sequence



Insufficient balance preserves the live hold. An expired hold returns `E_HOLD_EXPIRED`. No partial purchase or double debit.

Real e-ticket email delivery

Stage	Behavior
Payment commits	Booking is TICKETED. Store ticket codes and an email job
Worker sends	Use the saved booking contactEmail and a configured real provider
Delivery status	QUEUED, SENT on provider acceptance, or FAILED
Provider fails	Keep the paid booking. Retry the email job without another charge
Customer resends	`RESEND_TICKET(bookingCode)` uses the same saved recipient/codes

- Email only. Provider credentials remain on the server.
- A timeout may duplicate email on retry, never payment or tickets.
- R7 demonstration must show actual inbox receipt with matching booking/ticket codes.

Cancellation and simple refund rules

Booking condition	Result
Paid, at least 2 h before departure	Allow cancellation. Refund 100% of current fare
Paid, less than 2 h, not disrupted	Reject with `E_CANCEL_NOT_ALLOWED`
Disrupted paid booking, before departure	Refund 100% of current fare, even within 2 h
Live PENDING_PAYMENT	Release all holds. Refund zero

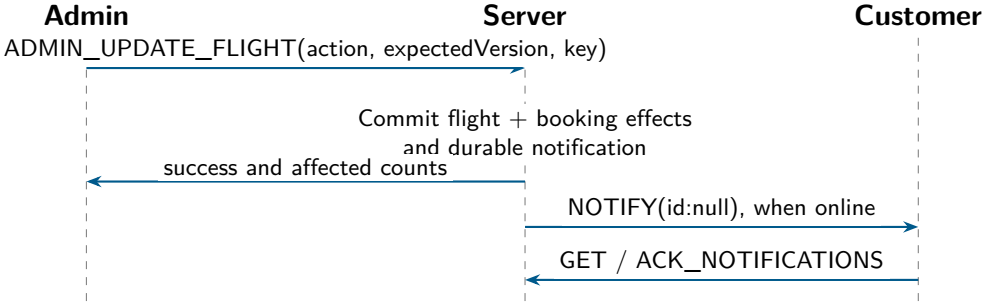
- Send bookingCode and idempotencyKey. No cancellation quote is required.
- Optional preview changes nothing. Apply checks current eligibility again.
- Cancellation fee is zero. Already departed paid bookings cannot cancel.

Viewing, changing and printing tickets

Operation	Procedure and outcome
View	LIST_BOOKINGS, then GET_BOOKING for current itinerary, status and tickets
Change	Select replacement on the same route/class. Preview a 60-second quote, then apply with quoteId/key
Settle change	Proposed fee 200,000 VND/passenger, waived if disrupted, plus signed fare difference
Update ticket	Swap seats atomically, increment revision and queue revised email. Preserve booking/ticket identifiers
Print	PRINT_TICKET returns current text lines. Client opens its print/save dialog

Change requires every current leg and replacement to depart at least 2 h later. Any failed change preserves the original itinerary and tickets.

Admin updates and customer notifications



DELAY/RESCHEDULE updates paid itineraries and invalidates unpaid holds. CANCEL releases all affected booking legs and refunds paid bookings once. Offline customers recover from the saved inbox.

Message processing on both ends

Server processing

1. Assemble and parse a bounded frame.
2. Log safely. Validate envelope, fields, role and ownership.
3. Check replay and business guards.
4. Commit inventory, balance, booking and saved result together.
5. Enqueue reply/push and write all bytes.

Client processing

1. Validate input and retain request ID/key.
2. Send one request and keep receiving.
3. Dispatch NOTIFY separately from replies.
4. Match response ID/type and show authoritative results/errors.
5. On timeout, reconnect, log in and reconcile.

One domain lock plus a SQL transaction serializes small-demo mutations. Socket writes and provider calls happen after release.

Failure class	Response and recovery
Invalid complete JSON frame	ERROR_RESP / E_BAD_JSON. Keep the connection usable
Missing login or wrong role	E_NOT_AUTHENTICATED or E_FORBIDDEN. No business mutation
Business rejection	Typed error, such as E_SEATS_UNAVAILABLE or E_INSUFFICIENT_BALANCE
Oversize / incomplete-frame timeout	Bound memory, log the fault and close that connection

- Log timestamp, peer/connection, user, direction, type/ID, byte count and result.
- Distinguish SEND_ATTEMPT, SEND_SUCCESS and SEND_FAILURE.
- Redact passwords, contacts and passenger identity data. Never log malformed raw payloads.

Recovery after a lost reply

Example: payment commits, but the connection closes before the reply arrives.

1. Client records an unknown outcome and keeps the original payload/key.
2. Reconnect and log in again with a fresh connection request ID.
3. Retry the same mutation with its original idempotencyKey.
4. Server returns the saved success before checking the current booking state.
5. Client refreshes the booking, balance and unread notifications.

Request ID	Correlates one exchange on one connection
Idempotency key	Identifies a committed operation across reconnect/restart

Validation and implementation acceptance

Current design evidence

- 92 synthetic frames cover all 19 request types.
- Validator checks correlation, payloads, replay and money arithmetic.
- 40 acceptance scenarios define future observable results.

Required runtime evidence

- Client on PC B communicates with server on PC A.
- Real email arrives with matching codes.
- Logs, last-seat races, lost replies and offline notifications behave correctly.

Next decisions: allowed server language, SQL engine and email provider. Implement, execute acceptance scenarios and record results for the later final report.

Questions

Appendix: account, search and comparison processing

Operation	Client sends	Server checks/actions	Client result
PING	Empty data	Read server time	Clock/liveness result
REGISTER	Username, password, name, email	Validate, hash password, enforce uniqueness, create USER	Show account result, then offer login
LOGIN / LOGOUT	Credentials / empty data	Bind stored identity / flush logout and close	Show role / return to disconnected
GET_BALANCE	Empty data	Read authenticated user's balance	Display authoritative VND balance
SEARCH_FLIGHTS	Route, dates, class, count, sorting	Check dates and eligibility, release stale holds, price/filter	Display offers or successful empty list
COMPARE_FLIGHTS	Flight IDs, class, count	Same route/date, live seats, calculate deltas	Show cheapest/fastest and differences

Appendix: purchase and email processing

Operation	Input	Server processing	Result / important errors
BOOK_FLIGHT	Selected legs, class, passengers, contactEmail, key	Replay, route/order, inventory. Hold all legs or none	PNR, revision, fare, expiry. Seat/unavailable errors
PAY_BOOKING	PNR, method, key	Replay, owner, live hold, flight status, balance. Debit/issue atomically	TICKETED, codes, balance, QUEUED email. Hold/balance errors
RESEND_TICKET	PNR only	Own TICKETED booking. Queue real email to saved contact	EMAIL, masked recipient, status. State/ownership errors

Client retains mutation keys across timeouts. Email failure never reverses a committed payment. A resend does not charge or issue another ticket.

Appendix: ticket-management processing

Operation	Client procedure	Server processing	Observable outcome
LIST / GET_BOOKING	Page own bookings, open PNR	Owner/filter/snapshot checks	Current detail, terminal history still viewable
CANCEL_BOOKING	Optional preview, then code/key	Replay, owner, live state, simple cutoff/disruption rule	Release all legs. Full fare or zero unpaid refund
CHANGE_BOOKING	Select leg/replacement, quote, accept/apply	Owner, quote/revisions, route, time, seats, balance	Atomic swap and signed settlement, revised email
PRINT_TICKET	Request text, choose local print/save	Own TICKETED booking, bounded current text	Client prints current revision

Terminal records reject active-ticket operations. Another user's booking appears as E_NOT_FOUND. Local print cancellation has no server effect.

Operation	Server behavior	Client behavior
ADMIN_LIST_FLIGHTS	ADMIN only. Filter/page current flight versions	Select flight and retain version
ADMIN_UPDATE_FLIGHT	Replay, active flight, expectedVersion, action fields. Commit all effects	Show saved counts, refresh affected data
NOTIFY	Save inbox entry, then push to online customer sockets	Deduplicate notificationId, show update, refresh booking
GET_NOTIFICATIONS	Return own complete entries with read flag, bounded pages	Recover unread updates after login
ACK_NOTIFICATION	Check all IDs owned, then mark read atomically	Update read state. Repeated ACK is safe

DELAY needs delayMinutes. RESCHEDULE needs new departure/arrival. CANCEL needs neither. Unknown action fields are rejected.

Apply directly, without a quote


```
{"v":1,"type":"CANCEL_BOOKING","id":"c-022",
 "data":{"bookingCode":"X4K9PL","dryRun":false,
 "idempotencyKey":"idem-cancel-1"}}
```


Refund the full current fare


```
{"v":1,"type":"CANCEL_BOOKING_RESP","id":"c-022",
 "ok":true,"code":"OK","data":{"dryRun":false,
 "bookingCode":"X4K9PL","status":"CANCELLED","fee":0,
 "feeReason":"INVOLUNTARY_CHANGE","refundAmount":1750000,
 "balanceAfter":4800000,"currency":"VND","revision":5}}
```


Each displayed JSON object occupies one transmitted line, followed by LF. The source transcript uses a fixed synthetic clock.

Appendix: full server push example


```
{"v":1,"type":"NOTIFY","id":null,"data":{
 "notificationId":"N-9001","kind":"DELAY",
 "bookingCode":"X4K9PL","flightId":"F1004","flightVersion":2,
 "flightNo":"VN215","oldDepartAt":"2026-12-20T14:00:00+07:00",
 "newDepartAt":"2026-12-20T16:30:00+07:00",
 "newArriveAt":"2026-12-20T18:40:00+07:00","delayMinutes":150,
 "freeCancellation":true,"message":"Flight delayed by 150 minutes",
 "createdAt":"2026-11-04T10:40:00+07:00"}}
```


- A 150-minute net shift marks the booking disrupted.
- Push uses no request ID. Stored inbox entries add a read flag.
- The customer explicitly acknowledges notification IDs after reading.

Field group	Contract
Money / time	Integer VND. ISO-8601 timestamps with +07:00 offset
Passengers / class	1–9 adults. ECONOMY, PREMIUM_ECONOMY or BUSINESS
Identifiers	PNR: six uppercase letters/digits. Request ID/key: 1–64 ASCII characters
Collections	Compare 2–5 flights. List page 1–20 rows. ACK 1–50 unique IDs
Validation	Reject duplicate JSON keys, non-finite numbers, unknown request fields and wrong types

`E_NOT_FOUND`, `E_STATE_CONFLICT`, `E_HOLD_EXPIRED`, `E_CANCEL_NOT_ALLOWED`, `E_QUOTE_EXPIRED`, `E_VERSION_CONFLICT`, `E_IDEMPOTENCY_CONFLICT`.

Appendix: flight states and admin effects

Trigger	Flight state	Booking effect
Initial schedule	SCHEDULED	Customers may hold eligible seats
DELAY	DELAYED, version increments	Update paid itinerary and notify. Invalidate unpaid holds
RESCHEDULE	RESCHEDULED, version increments	Same, with new absolute times
CANCEL	CANCELLED, terminal	Cancel whole affected live bookings and release all legs
Clock reaches departure	DEPARTED, derived	Reject new booking and post-departure cancellation/change

DELAY and RESCHEDULE may repeat on active future flights. A cancelled flight never becomes DEPARTED. Net shift from the accepted baseline determines disruption.

Appendix: source documents and review coverage

Source	Used for
Docs/topic.md	R1–R9, balance payment, mocked flights, admin triggers and user clarifications
Docs/Requirement.png	Introduction, architecture, functionality, protocol messages/states/processing
Lecture PDF pp.10,12–14,21–24	State/sequence/processing guidance, direct sockets, two PCs, design review and later evaluation
Design-Review/*.md	Current rules, use cases, wire formats, states, handlers and planned acceptance scenarios
message-catalog.json / wire-examples.jsonl	Operation inventory and synthetic examples

Review coverage: slides 2–3 introduction, 4 architecture, 5–6 functionality, 7–19 protocol design, 20 validation plan. Appendix tables cover all UC01–UC14.



THANK YOU !